Système et Réseaux : TD sérialisation

avril 2026 — Pierre Rousselin

Exercice 1 : Échanges UDP et appels système

Dans cet exercice, un client A envoie un datagramme contenant uniquement une chaîne de caractères « pas trop longue » (disons de 500 octets au plus) à un serveur B. Le protocole applicatif (bête, inventé pour cet exercice) CRIP (cri protocol) est qu'une fois que B a reçu la chaîne, il en change toutes les minuscules (ASCII) en majuscules et renvoie le résultat à A, trois fois de suite, chaque fois à une seconde d'intervalle.

1. Rappeler de quelles informations doit disposer A pour envoyer son datagramme à B, et fixer (arbitrairement, mais pas trop) ce qui doit l'être.
2. On rappelle ici (dans le désordre) le nom des appels systèmes vus en cours : `sendto`, `bind`, `close`, `recvfrom`, `socket`. Écrire, de façon informelle, les opérations dans l'ordre, côté serveur et côté client pour ce protocole. On dira simplement quelles informations sont nécessaires en entrée, et lesquelles sont données en sortie de ces appels. Le serveur est dans une boucle infinie.
3. Quel est le problème si des requêtes de clients arrivent, par exemple, toutes les secondes au serveur ? Comment le résoudre ?

--- * ---

Exercice 2 : Alignement et remplissage (*padding*)

Pour des raisons de performance, la mémoire d'un processus est en général organisée de telle sorte que les données de certains types respectent des *contraintes d'alignement*, c'est-à-dire sont stockées sur des adresses multiples d'une certaine puissance de 2. Ceci dépend à la fois de la machine et du compilateur utilisé (voire des options de compilation).

Voici un programme et sa sortie [1]

```c
#define FMT_ADDR "addresse de %s: %p\n"
int main(void)
{
        char c;
        uint32_t n;
        char d;
        uint16_t s;
        printf(FMT_ADDR, "c", &c);
        printf(FMT_ADDR, "n", &n);
        printf(FMT_ADDR, "d", &d);
        printf(FMT_ADDR, "s", &s);
}
```

```
addresse de c: 0x7ffd5e1e7f2f
addresse de n: 0x7ffd5e1e7f28
addresse de d: 0x7ffd5e1e7f27
addresse de s: 0x7ffd5e1e7f24
```

1. Représenter les octets numéro `...24` à `...2f` et dire lesquels correspondent à des variables et lesquels sont « perdus ».
2. Les contraintes d'alignement suivantes sont assez répandues sur les machines `x86-64`
   - Les entiers sur 1, 2, 4 et 8 octets sont alignés respectivement sur 1, 2, 4 et 8 octets.
   - Les `double` et les pointeurs sont alignés sur 8 octets.
   - Les structures sont alignées sur la plus grande contrainte d'alignement de leurs membres.

   En faisant ces hypothèses, pour chaque structure ci-dessous, donner sa contrainte d'alignement, sa représentation en mémoire (quels octets correspondent à du remplissage) et sa taille en octets.

1. La machine a pour architecture `x86-64`, le compilateur est `gcc` sans aucune option.

```
struct s1 {
        char s[3];
        int16_t i;
        char d;
        uint32_t n;
};
struct s2 {
        char c;
        double x;
        uint16_t k;
        int *p;
        char d;
};
```

--- * ---

Exercice 3 :

Un client et un serveur s'échangent des données sur des films. Pour le moment, supposons qu'un film est représenté par le type suivant (chez le client et le serveur) :

```
struct film {
        char nom[61];
        uint32_t duree; /* en minutes */
        uint16_t annee;
};
```

1. Pourquoi ne pas avoir utilisé les types `short` et `int` ?

2. Décrire les remplissages si les contraintes d'alignement sont les suivantes :

— les entiers sur 16 bits sont sur des adresses multiples de 2 ;

— les entiers sur 32 bits sont sur des adresses multiples de 4.

3. Quel est le problème dans le bout de code qui suit ?

```
struct film f = { .nom = "La mouche", .duree = 95, .annee = 1986 };
int sock = socket(AF_INET, SOCK_DGRAM, 0);
/* ... */
sendto(sock, &f, sizeof(struct film), 0, (struct sockaddr *) &s_srv, sizeof(s_srv));
```

4. Écrire une partie de protocole permettant d'envoyer et recevoir les informations contenues dans la structure, puis les fonctions de sérialisation/désérialisation :

```
void htonfilm(const struct film *src, uint8_t *dest);
void ntohfilm(const uint8_t *src, struct film *dest);
```

qui codent et décodent ces informations.

5. Problème : ceci est un titre de film.

Night of the Day of the Dawn of the Son of the Bride of the Return of the Revenge of the Terror of the Attack of the Evil, Mutant, Alien, Flesh Eating, Hellbound, Zombified Living Dead Part 2 : In Shocking 2-D

Quels changement faire dans `struct film` pour avoir plus de souplesse dans la longueur des titres ? (Ne pas perdre trop d'octets pour parler du film « Z », par exemple).

Modifier votre protocole en utilisant un nouveau champ (dans le message) pour la longueur du titre et aucun caractère délimiteur pour le titre. Modifier les fonctions `htonfilm` et `ntohfilm` en conséquence. Ce champ pour la longueur est-il nécessaire si le protocole sous-jacent est UDP ?

--- * ---